CO1 - AT 2

Concept Mapping

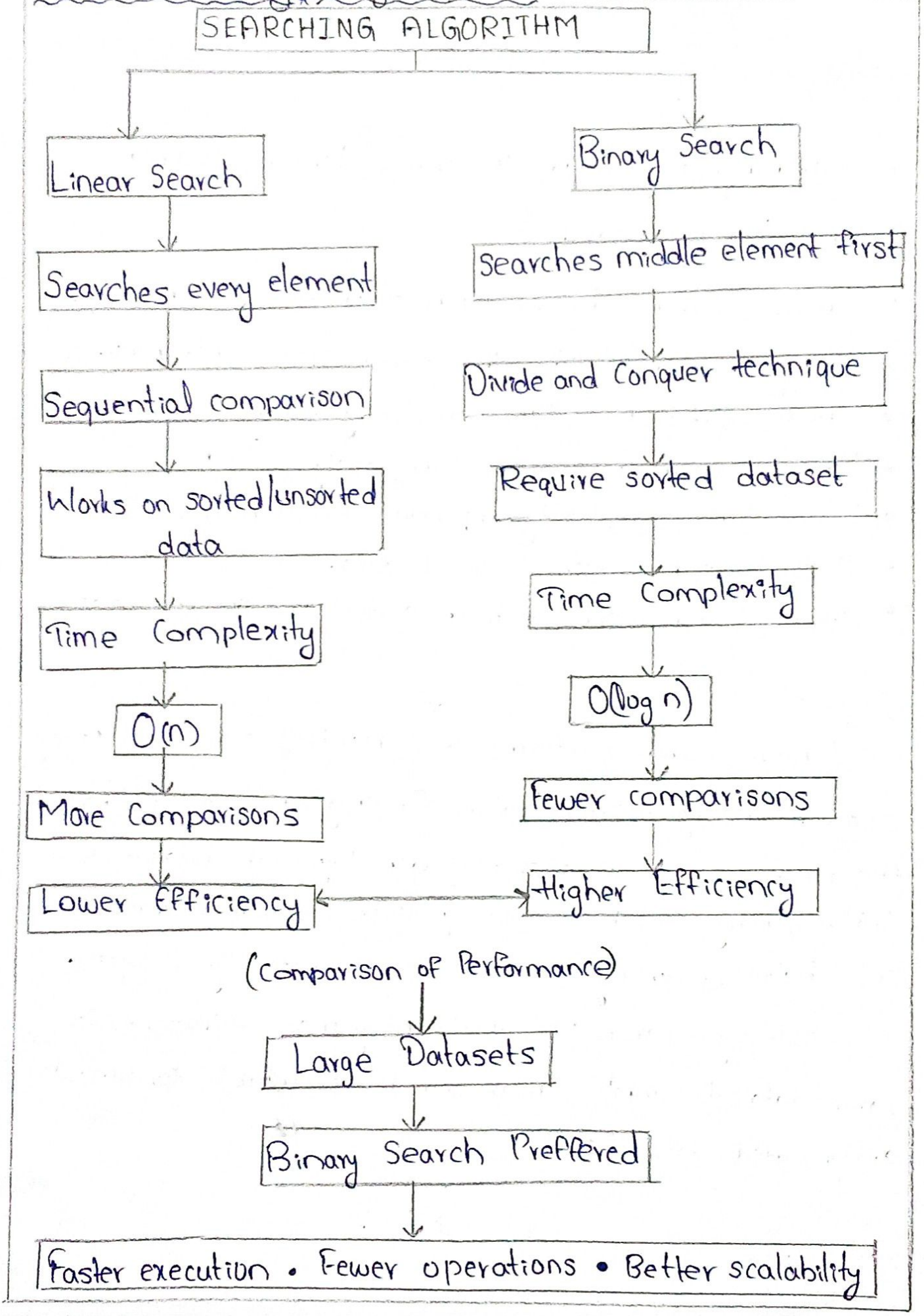
Name : P. Pushpa Charitha

Reg No. : 192525371

Course : Data Structures

Course code : CSA0312

Searching Algorithms



Relationships

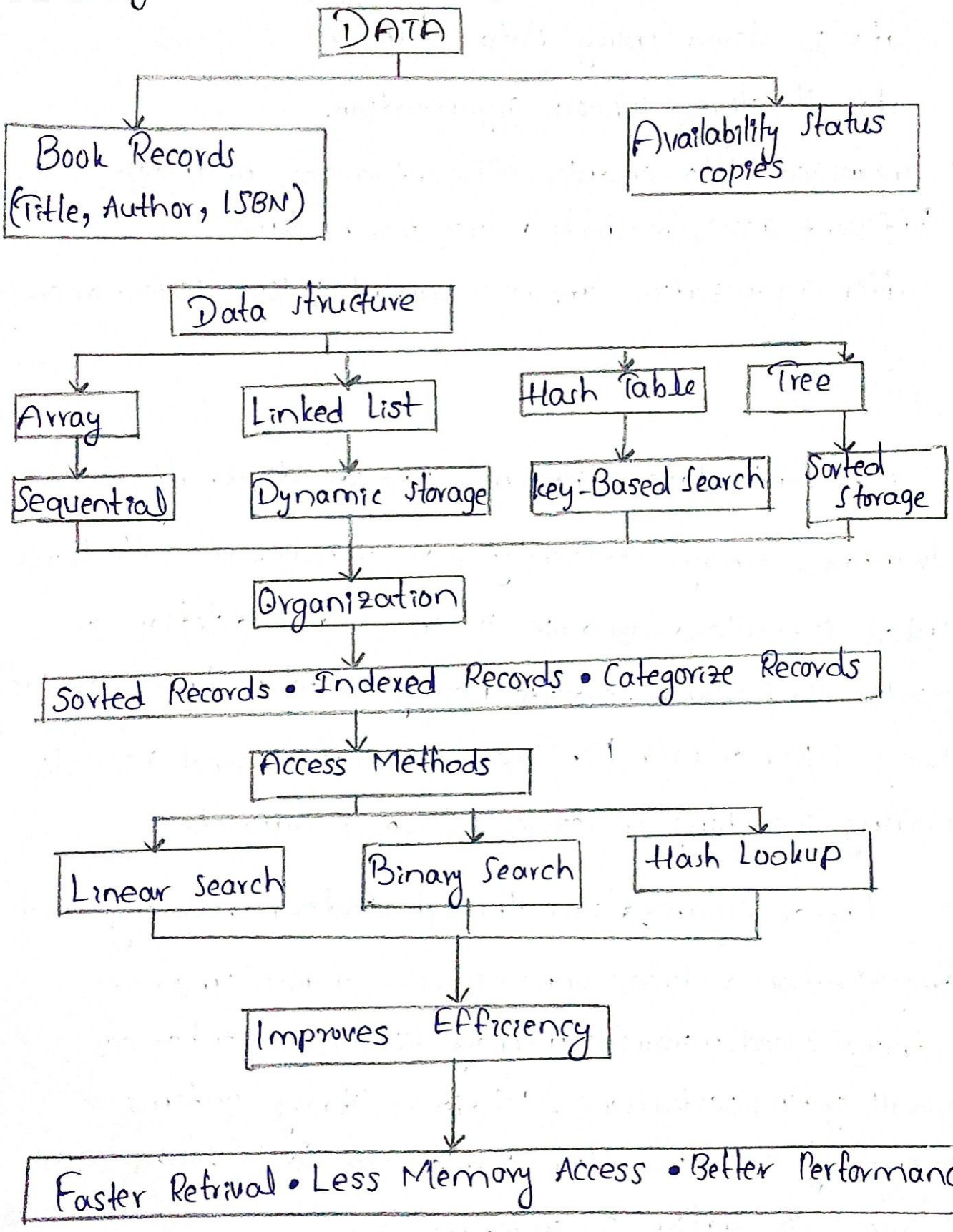
- Searching Algorithm includes Linear Search and Binary Search.
- Linear Search checks each element sequentially.
- Binary Search divides the search space into halves.
- Linear search has Time complexity O(n).
- Binary Search has Time complexity O(log n)
- Lower time complexity improves efficiency.
- Binary search requires sorted data.
- Linear search works on both sorted and unsorted data.

Explanation

Linear search performs a sequential scan and may examine every record before finding the target. Therefore, the no.of comparisons increases directly with dataset size O(n). Binary search repeatedly divides a sorted dataset into halves, reducing comparisons dramatically O(log n).

Binary search is significantly more efficient for large datasets, while linear search is suitable for small or unsorted datasets.

Library Management System



Relationships

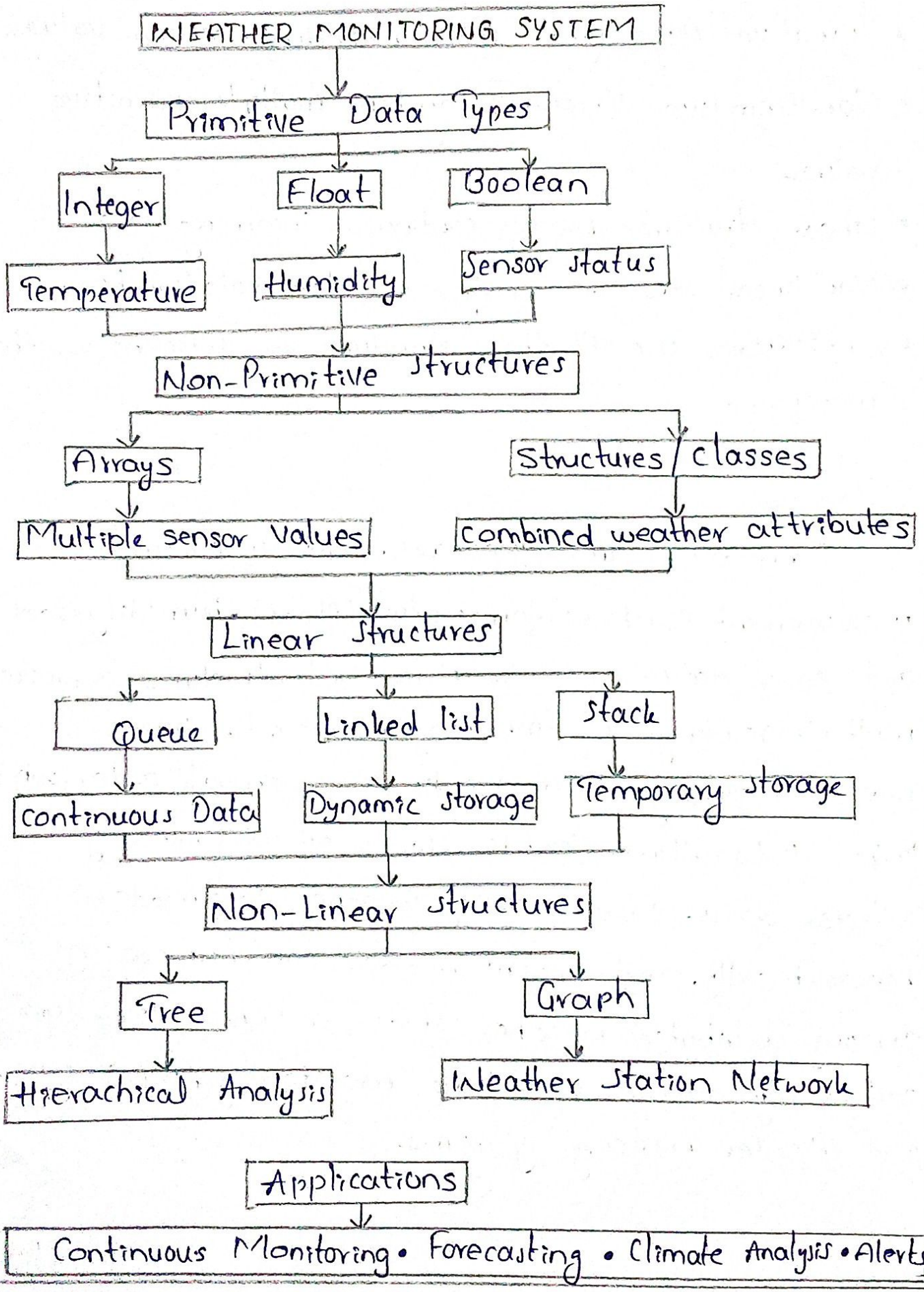
- Data is stored using Data structures.
- Data structures support organization.
- Organized data enables efficient access methods.
- Efficient access methods reduce search time.
- Better organization improves overall system performance.

Explanation

Different data structures provide different advantages. Arrays efficiently store fixed-size collections Linked lists allow dynamic insertion and deletion of records. Hash tables provide nearly constant-time searches using ISBN or book ID. Trees maintain sorted records, making searching efficient for large datasets.

Proper organization through indexing, sorting, and categorization reduces unnecessary searches, improves retrieval speed, minimize memory access, and enhances overall system performance. Modern library systems commonly combine Hash Tables for fast lookup and B-Trees for database indexing.

Weather Monitoring System



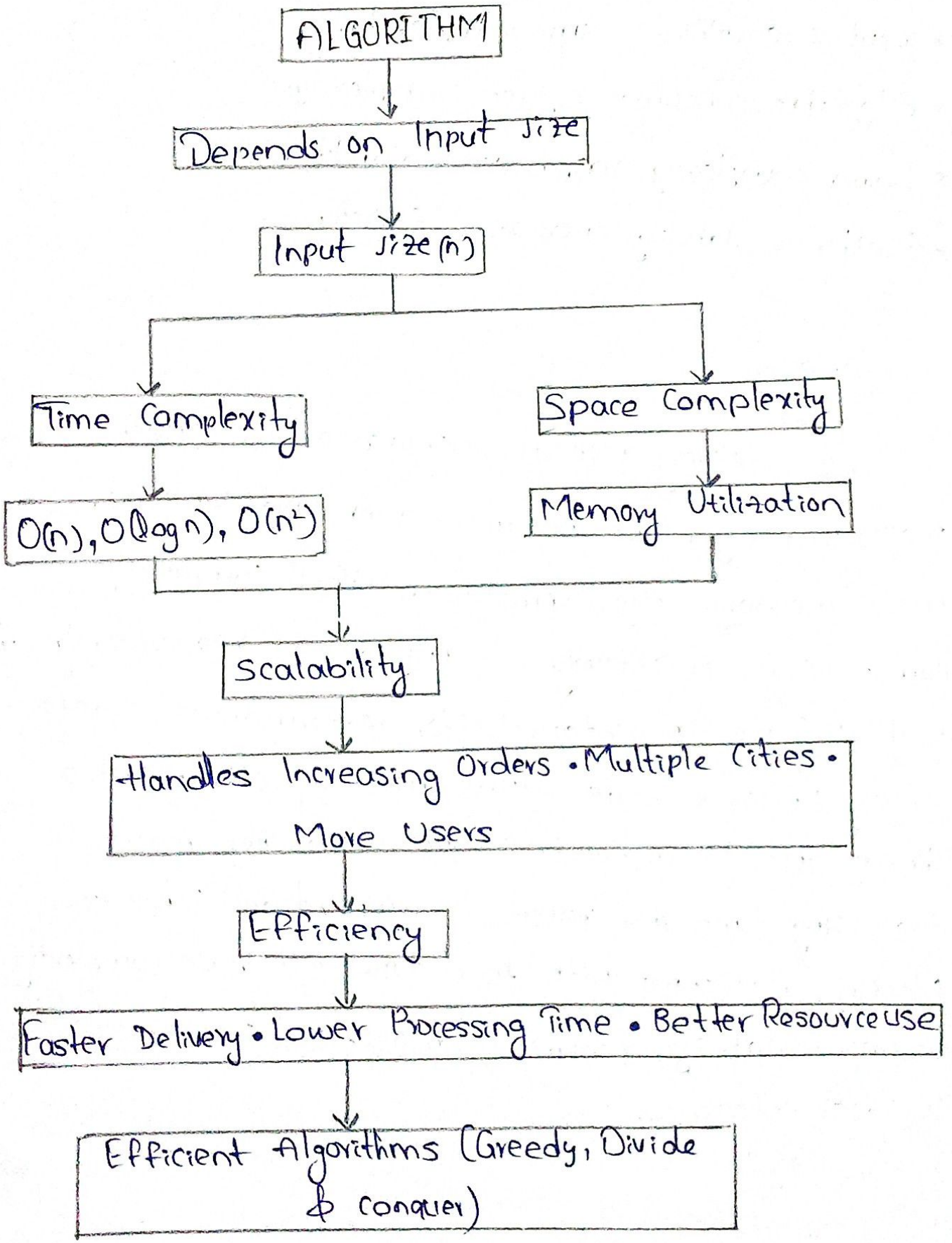
Relationships

- Primitive Data types store individual sensor values.
- Non-primitive structures combine multiple primitive values.
- Linear structures process continuous incoming data.
- Non-linear structures analyze complex relationships.
- Applications use all these structures for efficient weather monitoring.

Explanation

Primitive Data types represent individual measurements such as temperature (float), humidity (float), and sensor status (Boolean). Arrays and structures organite multiple readings into meaningful records. Queues process incoming sensor readings in arrival order, while linked lists allow flexible storage of continuously collected data. Trees classify weather information hierarchically, and graphs models communication among distributed weather stations. These structures collectively support real-time monitoring, forecasting, and disaster warning systems.

Logistics Company



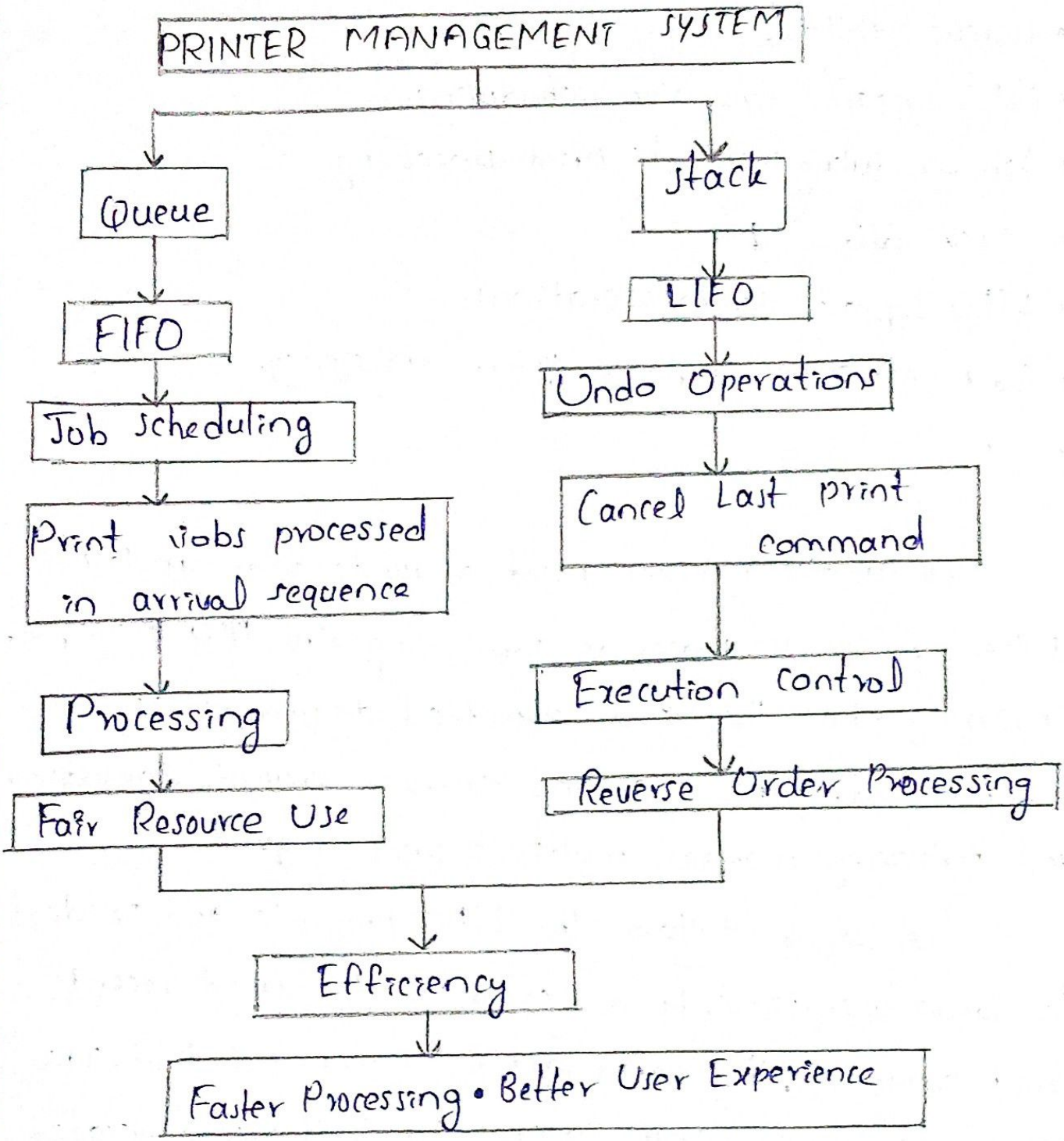
Relationships

- Input size affects Time complexity.
- Algorithm consumes Space complexity.
- Lower complexity improves Scalability.
- Better scalability increases Efficiency.

Explanation

As delivery requests increase across cities, inefficient algorithms become slower and consume more memory. Algorithms with $O(n^2)$ complexity cannot scale effectively for millions of requests. Algorithm with $O(\log n)$ maintain better performance as data grows. Efficient route optimization algorithms, such as Greedy and Divide-and-Conqure approaches, reduce processing time and resource consumption. Therefore, selecting algorithms with lower time and space complexity ensures scalability, minimizes delays, and improves logistics efficiency.

Printer System



Relationships

- Queue follows FIFO
- FIFO supports fair job scheduling.
- Job scheduling controls print processing
- Stack follows LIFO
- LIFO supports Undo operations.
- Both structures improve system efficiency.

Explanation

A printer receives print requests from multiple users. A queue processes jobs using the FIFO principle, ensuring that the first submitted documents is printed first. This guarentees fairness, prevents starvation and efficiency manages multiple print requests.

A stack follows the LIFO principle and is ideal for Undo operations. If a user cancels the most recent print command, the latest action is removed first. This mirrors how undo features operate in software applications.

Therefore :—
Queue is the most suitable data structure for print job scheduling because it guarantees fairness and orderly processing. Stack is the best choice for undo functionality because it naturally reverse the latest operation first.